database or via an injection attack. Read access allows the attacker to steal tokens and gain access to the API as any user. Write access allows them to create fake tokens or alter their own token. If they gain delete access, then they can delete other users' tokens, denying them access.

5.3.1 Hashing database tokens

Authentication tokens are credentials that allow access to a user's account, just like a password. In chapter 3, you learnt to hash passwords to protect them in case the user database is ever compromised. You should do the same for authentication tokens, for the same reason. If an attacker ever compromises the token database, they can immediately use all the login tokens for any user that is currently logged in. Unlike user passwords, authentication tokens have high entropy, so you don't need to use an expensive password hashing algorithm like Scrypt. Instead you can use a fast, cryptographic hash function such as SHA-256 that you used for generating anti-CSRF tokens in chapter 4.

Listing 5.9 shows how to add token hashing to the `DatabaseTokenStore` by reusing the `sha256()` method you added to the `CookieTokenStore` in chapter 4. The token ID given to the client is the original, un-hashed random string, but the value stored in the database is the SHA-256 hash of that string. Because SHA-256 is a one-way hash function, an attacker that gains access to the database won't be able to reverse the hash function to determine the real token IDs. To read or